School of Electrical Engineering and Computer Science
INFS3208 – Cloud Computing

Programming Assignment Task I (10 Marks)

Task description:

Your team manager requires a website solution that can promptly deploy your company's PHP-based web pages using Nginx as the webserver and PHP for server-side scripting, all encapsulated within a single container. As a developer, your task would be to create a Dockerfile to containerize these applications. This Dockerfile will then be shared with the PHP developers who are working on the website. They should have the capability to upload their PHP websites to a remote directory within the container and immediately conduct testing without encountering any issues. The Dockerfile you create must adhere to the following technical requirements:

1. **You are NOT permitted to directly pull existing Docker images from Dockerhub.com** (for instance, webdevops/php-nginx). The only base image you are allowed to use is Ubuntu 20.04 (Long Term Support) (for instance, ubuntu:20.04) and you need to build your image from this base.
2. (**1 mark**) To ensure that the creator of the Docker image can be contacted by your colleagues, you must provide the related information (your name and email), using the appropriate Dockerfile instruction. Additionally, you should define a variable for the PHP version (i.e., PHPVER = 8.2) and the time zone so that the specific version of PHP can be installed.
3. (**1 mark**) Nginx can deal with static web pages (e.g., html) but it cannot directly process PHP pages. Thus, it needs FastCGI Process Manager (FPM or PHP-FPM) to process PHP pages. You need to **install Nginx, and PHP-FPM** with a specified version to work for PHP websites. (**0.5 marks**) To communicate between Nginx and PHP-FPM, the settings of Nginx are crucial for supporting PHP-FPM. In your Dockerfile, ensure you can **copy the modified Nginx setting file** (provided in cca1 folder) **to replace the default one** (`/etc/nginx/sites-available/default`) in the container by using a Dockerfile instruction.
4. (**1 mark**) The testing html and php pages are available on GitHub (`https://github.com/csenw/cca1`), you need to download them and **copy them to the root folder** (/var/www/html) of the website in the container with the learned instructions. They must be functional once the container is running.
5. (**2.5 marks**) To enable access from the outside, you need to (1) **expose and publish port 80 for Nginx**, (2) **designate the root folder of the website as a volume**, (3) **set the working directory to** (/var/www/html), (4) **start Nginx and PHP-FPM with the entrypoint command and make sure nginx running in the foreground**. With these steps, PHP files in the root folder will sync instantly with any changes in the PHP files in the host's mounted folder. This eliminates any need for PHP programmers to interact further with the container.
6. (**2 marks**) You need to **follow the good practices** of writing a Dockerfile making the image as lean as possible.
7. (**2 marks**) To correctly build the image from the Dockerfile and run the container, you need to **provide related docker commands** with appropriate options (e.g., *docker build [options]* and *docker run [options])* in a separate text file. **Tag** the built image as 'a1:<your surname>' and the container as 'cca1_nginx_php'.

Preparation:

In this individual coding assignment, you will put your Docker command skills, Dockerfile instruction knowledge (from Lecture 4), and relevant fields to practical use. The first step is to comprehend the task and understand the technical specifications required. Next, you should utilize Linux commands to install Nginx, PHP, and PHP-FPM on a Virtual Machine (VM), following the guidance provided in the Practical/Tutorial sessions. Lastly, these Linux commands need to be transformed into Dockerfile instructions to accomplish the task at hand. You can practice on either the VM from Google Cloud Platform (GCP) or your local machine (requiring Oracle VirtualBox), depending on your access to GCP. Please refer to the Dockerfile writing example below for more detailed instructions.

Assignment Submission:

- You must compress the **Dockerfile** and **a text file** that contains docker commands to properly build the image and run it.
- The name of the compressed file should be named as "FirstName_LastName_StudentNo.zip".
- You must make an online submission to Blackboard **before 1:00 PM on Friday, 01/09/2023**.
- Only one extension application could be approved due to medical conditions.

Main Steps:

Step 1:

Log in your VM and change to your home directory.

Step 2:

```
git clone https://github.com/csenw/cca1.git && cd cca1
```

Run this command to download the required configuration files and testing web pages.

```
uqteaching@instance-1:~$ git clone https://github.com/csenw/cca1.git && cd cca1
Cloning into 'cca1'...
remote: Enumerating objects: 23, done.
remote: Counting objects: 100% (23/23), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 23 (delta 4), reused 10 (delta 0), pack-reused 0
Unpacking objects: 100% (23/23), 13.64 KiB | 1.95 MiB/s, done.
uqteaching@instance-1:~/cca1$
```

In this folder (cca1), please **write your Dockerfile**.

Step 3:

Build your Dockerfile and tag your image as a1:<your surname>.

```
 ---> df8ca524271d
Successfully built df8ca524271d
Successfully tagged a1:wang
```

Step 4:

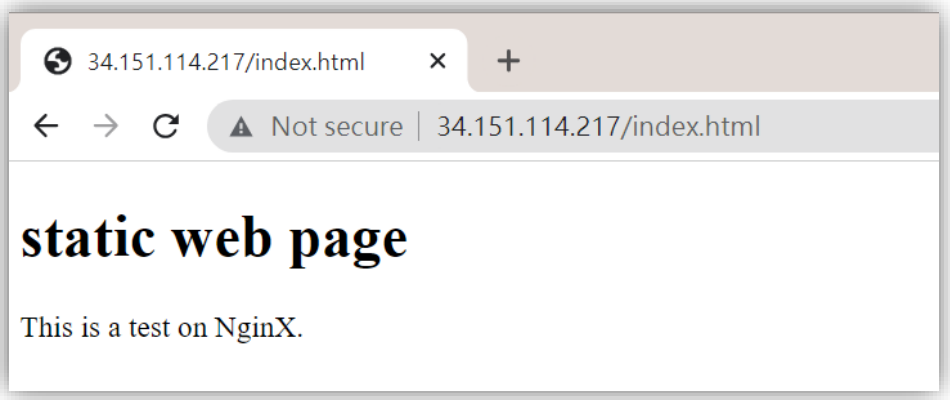
Run the image as a container ‘cca1_nginx_php’:

```
uqteaching@instance-1:~/cca1$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS         PORTS                                   NAMES
f976ae62abea   a1:wang   "/bin/bash -c '/usr/…"   2 minutes ago  Up 2 minutes   0.0.0.0:80->80/tcp, :::80->80/tcp       cca1_nginx_php
```

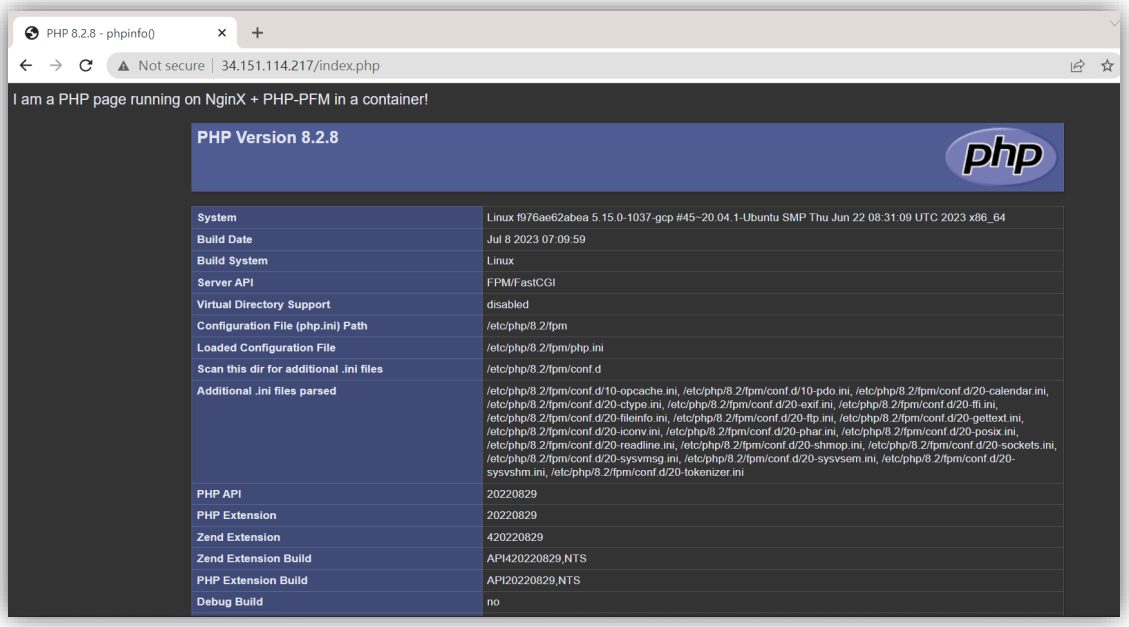
Step 5:

Test the container on your VM with the preloaded index.html and index.php.

Static webpage test: `http://public_ip/index.html`



PHP webpage test: `http://public_ip/index.php`



Appendix

Nginx (pronounced "engine X"), stylized as NGINX or Nginx or NginX, is a web server that can also be used as a reverse proxy, load balancer, mail proxy and HTTP cache. The software was created by Igor Sysoev and publicly released in 2004. Nginx is free and open-source software, released under the terms of the 2-clause BSD license. A large fraction of web servers uses NGINX, often as a load balancer. Nginx can be deployed to serve dynamic HTTP content on the network using FastCGI, SCGI handlers for scripts, WSGI application servers, and it can serve as a software load balancer. Note that Nginx cannot directly deal with PHP but can serve PHP applications through the FastCGI protocol. Nginx employs PHP-FPM (FastCGI Process Manager) that is running in the background as a daemon and listening for CGI requests. Nginx uses an asynchronous event-driven approach, rather than threads, to handle requests. Nginx's modular event-driven architecture can provide more predictable performance under high loads. Nginx default configuration file is nginx.conf

PHP [2] is a general-purpose scripting language that is especially suited to web development. It was created by Danish-Canadian programmer Rasmus Lerdorf in 1994; the PHP reference implementation is now produced by The PHP Group. PHP originally stood for Personal Home Page, but it now stands for the recursive initialism PHP: Hypertext Preprocessor. PHP code is usually processed on a web server by a PHP interpreter implemented as a module, a daemon or as a Common Gateway Interface (CGI) executable. On a web server, the result of the interpreted and executed PHP code – which may be any type of data, such as generated HTML or binary image data – would form the whole or part of an HTTP response. Various web template systems, web content management systems, and web frameworks exist which can be employed to orchestrate or facilitate the generation of that response. Additionally, PHP can be used for many programming tasks outside of the web context, such as standalone graphical applications and robotic drone control. Arbitrary PHP code can also be interpreted and executed via the command-line interface (CLI).

PHP-FPM (FastCGI Process Manager) [2] is an alternative FastCGI implementation for PHP, bundled with the official PHP distribution since version 5.3.3. When compared to the older FastCGI implementation, it contains some additional features, mostly useful for heavily loaded web servers. Figure 1 shows how PHP-FPM helps Nginx process PHP pages.

Figure 1.

How PHP and Nginx work together (Image credit: DataDog)

[1] Nginx, https://en.wikipedia.org/wiki/Nginx
[2] PHP, https://en.wikipedia.org/wiki/PHP